

KAFRIADA CORE — Pilot Build Specification

KOWAGURU TECHNOLOGY LIMITED · ENGINEERING · CONFIDENTIAL

Technical Build Specification v1.0

Owner: CTO · 18 August 2026 · Source: KAF 000 GDOC v1.0 + CEO decisions of 18 Aug 2026 · Audience: developers and coding agents. Each numbered section is written to be self-contained enough to hand to an agent as a work brief.

Product in one line: free athlete registration issuing a permanent KUID + public QR profile; paid Stage-2 verification (₦2,500 athlete / ₦15,000 club) unlocking photo and badge; free club rosters; free transfer *records* (no in-system transfer money); an admin-posted news feed with state/LGA targeting; a closed-loop wallet ledger. Pilot: Jigawa state, 27 LGAs, adults (18+) only.

Non-negotiables: KUID is immutable and never reused. Money enters only via Paystack. Wallet has no withdrawals and no P2P. All money in kobo integers. Every critical action writes an audit row. Webhook processing is idempotent.

1. Stack & repository layout

LAYER	CHOICE	NOTES
App	Next.js 15+ (App Router, TypeScript), single repo, UI + API routes together	Installable PWA (manifest + service worker for shell caching). One deploy unit.
ORM / DB	Prisma + PostgreSQL 16 (managed: Neon/RDS)	All schema changes via migrations, never manual SQL in prod.
Auth	Session cookies (httpOnly, secure, SameSite=Lax), argon2id password hashing	No JWT-in-localStorage. Re-auth for sensitive admin actions.
Payments	Paystack (initialize + webhook + verify API)	The only money rail. Test keys until week 7.
SMS	Termii (OTP only)	Email via Resend/SES for receipts and notices.
Media	S3-compatible (Cloudflare R2), presigned upload URLs	Max 10MB images, 50MB video. Never through the app server.
Edge / hosting	Cloudflare (DNS, CDN, WAF) → app on Render/Railway/EC2 (2vCPU/4GB)	~\$60–120/mo total pilot infra.

Module folders (no module imports another module's DB tables directly; cross-module calls go through exported service functions): `modules/identity` , `modules/clubs` , `modules/transfers` , `modules/feed` , `modules/wallet` , `modules/admin` , plus `core/` (auth, rbac, locations, notifications, audit, media).

2. Roles & access

ROLE CODE	CAN	CANNOT
<code>athlete</code>	Own profile CRUD, start Stage 2, accept/decline roster invites and transfers, view own wallet	See other athletes' contact/medical data
<code>club_admin</code>	Register club, manage teams/rosters, invite by KUID, list transfers, confirm incoming transfers	Edit athlete profiles, see wallets other than club's own
<code>coach</code>	View assigned team rosters, mark availability	Financial or transfer actions
<code>scout</code>	Browse/search verified (stage ≥ 2) athletes read-only	Contact data export; unverified profiles beyond name+KUID

<code>lga_coordinator</code>	View clubs/athletes in own LGA, initiate assisted payments (tagged with own id), onboarding stats	Approve verifications, edit records
<code>state_coordinator</code>	Same as LGA coordinator, state-wide, read-mostly	Approvals
<code>super_admin</code>	Everything: verification queue, club approvals, transfer validation, news CMS, refunds/reversals, role grants	Delete audit rows (nobody can)
<code>public</code>	View QR profile pages and public news (no login)	Everything else

RBAC is table-driven (`roles` , `user_roles`); permission checks are middleware on every API route — never UI-only. The GDOC's full 28 roles are future rows in `roles` , not future code.

3. Data model (PostgreSQL)

Conventions: `id uuid pk default gen_random_uuid()` , `created_at timestamptz default now()` on every table (not repeated below). Money = integer kobo. Enums as Postgres enums. Soft-delete nowhere in the pilot except `users.status` .

users

COLUMN	TYPE / CONSTRAINT	NOTES
<code>email</code>	citext unique nullable	CHECK: at least one of email/phone present. Duplicate check across both → 409.
<code>phone</code>	text unique nullable (E.164)	
<code>password_hash</code>	text not null	argon2id
<code>dob</code>	date not null	App-level gate: reject if age < 18 (pilot rule)
<code>full_name</code>	text not null	
<code>status</code>	enum: active suspended deactivated	suspended blocks login

roles / user_roles

COLUMN	TYPE / CONSTRAINT	NOTES
<code>roles.code</code>	text unique	Seed the 8 codes from §2
<code>user_roles</code>	pk (user_id, role_id); granted_by uuid fk users	Role grants audited

locations

Seed: NG → Jigawa (code JG) → 27 LGAs with 3-letter KUID codes (e.g. Birnin Kudu = BKD). Structure supports all states later.

COLUMN	TYPE / CONSTRAINT	NOTES
<code>kind</code>	enum: country state lga	
<code>name</code>	text	
<code>kuid_code</code>	text	unique within (kind, parent_id)
<code>parent_id</code>	uuid fk locations nullable	lga→state→country

athletes

COLUMN	TYPE / CONSTRAINT	NOTES
user_id	uuid fk users, unique	1:0..1 with users
kuid	text unique not null	Never updated. No update path in code. No reuse.
gender	enum: male female	
state_id / lga_id	uuid fk locations	Drives KUID codes + feed targeting
primary_sport	text not null (config list)	secondary_sport nullable
position	text nullable	Sport-specific, config-driven
dominant_side	enum: left right both, nullable	
years_experience	int nullable	
stage	smallint default 1	1 = registered, 2 = verified
photo_media_id	uuid fk media_files nullable	Rendered publicly only when stage ≥ 2 (silhouette otherwise)
visibility	enum: public verified_only private, default public	

kuid_counters

COLUMN	TYPE / CONSTRAINT	NOTES
year	int	pk (year, state_code). Row is SELECT ... FOR UPDATE'd inside the Stage-1 transaction; serial increments atomically.
state_code / last_serial	text / int default 0	

organizations / teams / roster_members

COLUMN	TYPE / CONSTRAINT	NOTES
organizations.name	text not null	type enum: club academy school other
organizations.state_id / lga_id	uuid fk locations	rep_user_id fk users (the club_admin)
organizations.status	enum: pending_review approved suspended	Only approved orgs build rosters
organizations.stage	smallint default 1	2 after ₦15,000 verification
teams.org_id	uuid fk organizations	name, sport, age_category (Senior/U23/...), gender
roster_members	team_id fk, athlete_id fk, jersey_no int null	status enum: invited active released. Partial unique index: one <i>active</i> row per (team_id, athlete_id). invited_by fk users.

verification_requests

COLUMN	TYPE / CONSTRAINT	NOTES
athlete_id	uuid fk athletes (nullable if org)	org_id fk organizations nullable — exactly one of the two set
target_stage	smallint (2)	Stage 3 = post-pilot, schema-ready
status	enum: draft payment_pending under_review approved rejected	Transitions in \$5 only
payment_id	uuid fk payments nullable	Set by webhook handler
reviewer_id / reviewed_at / rejection_reason	uuid fk users / timestampz / text	Rejection requires a reason; resubmit reuses the same payment (no second fee)

payments

COLUMN	TYPE / CONSTRAINT	NOTES
paystack_ref	text unique not null	Idempotency key — webhook replay must no-op
purpose	enum: stage2_athlete stage2_org	
amount_kobo	int not null	From config: 250000 / 1500000. Verify amount in webhook against expected.
status	enum: pending success failed abandoned	Redirect never sets success — only webhook or verify-API
paid_by_user_id	uuid fk users	Who initiated
on_behalf_of_user_id	uuid fk users nullable	Assisted payment target (the athlete)
coordinator_id	uuid fk users nullable	Set when assisted — feeds coordinator performance stats
raw_webhook	jsonb nullable	Store the verified payload

wallets / wallet_entries

COLUMN	TYPE / CONSTRAINT	NOTES
wallets.user_id	uuid fk users unique	balance_kobo int default 0 — updated only in the same tx as an entry insert
wallet_entries.direction	enum: credit debit	Append-only: no UPDATE or DELETE, ever. Corrections = reversing entries.
wallet_entries.source	enum: paystack fee reward reversal	payment_id fk nullable; memo text

transfers

COLUMN	TYPE / CONSTRAINT	NOTES
athlete_id / from_org_id	uuid fk, not null	to_org_id nullable until a buyer confirms
status	enum: listed player_accepted buyer_confirmed validated completed cancelled	Transitions in \$5 only. No fee fields in pilot — money is offline by design.
listed_by / validated_by	uuid fk users	validated_by = super_admin or recognized org officer

news_posts / career_events / media_files / notifications / audit_logs / otp_codes

TABLE	KEY COLUMNS	NOTES
news_posts	title, body (rich text), category enum: transfer results event sponsor announcement; tier enum: global state lga; target_state_id / target_lga_id fk nullable; pinned bool; published_at, expires_at	Feed query: global U user's state U user's LGA, pinned first, newest next
career_events	athlete_id fk, kind enum: registered verified joined_club released transfer, ref_id uuid, occurred_at, data jsonb	Append-only. Written automatically by identity, clubs, transfers modules — this is the GDOC career timeline
media_files	owner_user_id fk, kind enum: profile_photo action_photo video document logo, storage_key, mime, size_bytes, status enum: pending active	Presigned PUT; server confirms + activates after HEAD check
notifications	user_id fk, type, title, body, read_at nullable	In-app; email mirror for payment/verification events
audit_logs	actor_id fk nullable, action text, object_type, object_id uuid, ip inet, user_agent, result enum: ok denied error	Insert-only; no app-level update/delete path. Required events list in §7
otp_codes	target (phone/email), code_hash, purpose enum: register login reset, expires_at (now()+5min), attempts int, consumed_at	Max 3 attempts; resend cooldown 60s; rate-limit per target and per IP

4. KUID generation (module: identity)

Format: KA-{COUNTRY}-{STATE}-{LGA}-{YEAR}-{SERIAL6} → e.g. KA-NG-JG-BKD-2026-000123 . Codes come from `locations.kuid_code` ; YEAR = registration year; SERIAL6 = zero-padded counter from `kuid_counters` keyed (year, state_code).

Rule: generated inside the same DB transaction that completes Stage-1 registration (`SELECT ... FOR UPDATE` on the counter row → increment → insert athlete). A KUID can never exist without its athlete row; a retried request can never mint two. Public profile URL: `/a/{kuid}` ; the QR encodes `/a/{kuid}?s={HMAC-SHA256(kuid, QR_SECRET) first 16 hex}` — page renders a "genuine QR" tick only when the signature checks out.

5. State machines (enforce in service layer; invalid transition → 409 + audit row)

5.1 Verification

FROM	EVENT	TO	SIDE EFFECTS
—	Stage-1 registration completes	(athlete.stage = 1)	KUID minted, QR live, wallet created, career_event registered
draft	Athlete submits Stage-2 form (photo, height, weight, history) + taps pay	payment_pending	Paystack initialize; payment row pending
payment_pending	Webhook charge.success (verified sig + amount)	under_review	Payment success ; wallet credit + fee debit; enters admin queue; email receipt
payment_pending	72h elapsed unpaid (daily job)	draft	Payment abandoned
under_review	Reviewer approves	approved	athlete.stage = 2; photo public; badge on; career_event verified ; notify
under_review	Reviewer rejects (reason required)	rejected	Notify with reason
rejected	Athlete corrects + resubmits	under_review	No second fee — same payment_id carries over

5.2 Transfer record

FROM	EVENT	TO	SIDE EFFECTS
—	Selling club_admin lists own rostered player	listed	Notify player
listed	Player consents in-app	player_accepted	Notify seller; discoverable to clubs
listed / player_accepted	Player declines or club withdraws	cancelled	
player_accepted	Buying club (must hold approved KAFRIADA org) confirms	buyer_confirmed	to_org_id set — <i>this step recruits the buying club</i>
buyer_confirmed	super_admin / recognized org validates	validated	
buyer_confirmed	Validation refused	cancelled	Reason required
validated	System completes	completed	Old roster row → released; invite created at buyer; career_event transfer ; auto news_post (category transfer, tier lga/state)

6. Payments & wallet rules (module: wallet)

- Initialize via Paystack with metadata {purpose, user_id, on_behalf_of_user_id?, coordinator_id?} ; store **pending** payment first, then redirect/USSD code.
- Webhook is the source of truth.** Validate **x-paystack-signature** (HMAC-SHA512 of raw body with secret key); check **paystack_ref** not already **success** (idempotency); check amount matches expected for purpose; then in ONE transaction: payment→success, wallet credit entry, fee debit entry, balance update, verification_request→under_review, audit row.
- Daily reconciliation job: every **pending** older than 1h → Paystack verify API → settle or mark failed. Log divergences.

- No withdrawal endpoint, no P2P endpoint — they must not exist even disabled. Reward credits: super_admin only, audited, reason required.
- Assisted payment: coordinator initiates for athlete's account; entries land on the *athlete's* wallet; coordinator_id tagged on payment for ops stats.

7. API surface (all under /api ; middleware: session → role check → rate limit → handler; audit on every mutation)

ROUTE	METHOD	ROLE	PURPOSE
/auth/register , /auth/otp/verify , /auth/login , /auth/logout , /auth/reset	POST	public	OTP-verified signup (18+ DOB gate), sessions
/athletes/me	GET/PATCH	athlete	Profile; PATCH cannot touch kuid/stage
/athletes/me/verification	POST	athlete	Submit Stage-2 → returns Paystack init
/a/{kuid} (page) + /api/public/athletes/{kuid}	GET	public	QR profile; photo only if stage ≥ 2; QR signature check
/orgs , /orgs/{id} , /orgs/{id}/teams	POST/GET/PATCH	club_admin	Club reg (→ pending_review), teams
/teams/{id}/invites (by KUID), /invites/{id}/respond	POST	club_admin / athlete	Roster building; accept/decline
/transfers + /transfers/{id}/accept confirm validate cancel	POST	per §5.2	Transfer record lifecycle
/feed?scope=auto	GET	any logged-in / public subset	Targeted news feed
/wallet , /wallet/entries	GET	owner	Ledger view
/payments/webhook/paystack	POST	signature only	\$6. Excluded from CSRF; raw body preserved
/media/presign , /media/{id}/confirm	POST	owner	Uploads
/admin/verifications (+ approve/reject), /admin/orgs (+ approve), /admin/news CRUD, /admin/rewards , /admin/reports , /admin/audit	various	super_admin	Queues, CMS, reports; coordinator read views under /coord/*

8. Security & ops floor (launch blockers, all of them)

- Rate limits: OTP send 1/60s + 5/day per target; login 10/hr per account+IP; payment init 10/day per user.
- RBAC asserted server-side on every route (test: each role against each route matrix — deny by default).
- Audit events (minimum): register, login-fail, role-grant, verification submit/approve/reject, org approve, roster change, transfer every transition, payment every transition, reward credit, news publish, admin login.
- Backups: daily automated Postgres snapshot, 30-day retention, **one rehearsed restore before launch**. R2 media versioning on.

- TLS everywhere; secrets in env (never in repo); Paystack live keys only in prod env; error alerting (Sentry) to team phones; uptime check on `/api/health` .
- NDPR basics: privacy notice at signup, data-export and delete-request handled manually via support in pilot, 7-year audit retention per GDOC.

9. Eight-week plan with acceptance criteria

WK	SCOPE	DONE MEANS
1	Repo, CI, deploy pipeline, auth+OTP, RBAC, locations seed (JG + 27 LGAs + codes), audit log	Deploy on push; register/login works on staging; audit rows written
2–3	Athlete profile, KUID generation, media upload, public QR page	Concurrent-registration test mints unique KUIDs; QR scan on a real phone shows correct page; unverified = silhouette
3–4	Clubs: registration→approval, teams, invite-by-KUID, rosters	End-to-end demo: club registers, admin approves, invites 15 athletes, roster renders. Coordinator training build ships
5	Paystack init/webhook/reconcile, Stage-2 flow, wallet ledger, review queue	Test-mode payment moves request to under_review; replayed webhook no-ops; ledger balances to the kobo
6	Transfers end-to-end, news feed + targeting, notifications	Completed transfer updates both rosters + timeline + auto news post; LGA-targeted post visible only in that LGA
7	Hardening: rate limits, role×route matrix test, load test (500 concurrent), restore drill, copy review (simple English / Hausa-friendly)	Zero criticals; restore rehearsed; p95 < 500ms at pilot load
8	Prod deploy, monitoring, live Paystack keys, Birnin-Kudu soft launch, LGA rollout	First 100 real registrations without a P1

Ownership: Dev A — identity + verification + payments. Dev B — clubs + transfers + feed. Dev C — UI/PWA + admin portal + QA. CTO reviews every migration and everything touching money or KUID.

10. Configuration (env)

`DATABASE_URL` , `SESSION_SECRET` , `QR_SECRET` , `PAYSTACK_SECRET_KEY` , `PAYSTACK_PUBLIC_KEY` , `TERMII_API_KEY` , `R2_*` (endpoint/bucket/keys), `EMAIL_API_KEY` , `FEE_STAGE2_ATHLETE_KOBO=250000` , `FEE_STAGE2_ORG_KOBO=1500000` , `PILOT_MIN_AGE=18` , `SENTRY_DSN` . Prices and age gate are config, not code.

11. Open decisions (tracked; defaults apply if unconfirmed)

1. Wallet = closed-loop ledger (default: yes, per CEO risk discussion).
2. Prices ₦2,500 / ₦15,000 (default: yes, env-configurable).
3. 8-week timeline with week-4 coordinator build (default: yes).
4. Founding agreements signed before Sprint 0 (blocking).
5. GDOC chapter renumbering incl. O4 gap (default: renumber).

KAFRIADA CORE Pilot Build Specification v1.0 · CTO Office · Changes to this spec require a version bump and a note in the decision log. Sections 3–7 are safe to hand to a coding agent verbatim as work briefs.